Training Conversation: Multi-Agent Architectures in Microsoft Foundry Agent Service and Microsoft Agent Framework

Technical comparison, traffic patterns, cost/latency implications, and practical design guidance

Purpose: This Word document captures the whole training conversation in a cleaned-up format that can be used as instructor material or a student handout.

Scope: The document explains the difference between multi-agent use in Microsoft Foundry Agent Service and Microsoft Agent Framework (MAF), with special attention to LLM-to-tool traffic.

1. Original User Questions

1. Give me technical information about the difference to use multi-agents in Foundry AI Agent and MAF.
2. Give me details about how much the traffic between LLMs and the tools for each of them.
3. Give me the whole conversation in a Word document that can be used for training.

2. Executive Summary

Foundry Agent Service and Microsoft Agent Framework solve related but different problems. Foundry Agent Service is best understood as a managed enterprise platform/runtime for agents. MAF is best understood as a code-first framework for creating and orchestrating complex agent workflows.

```
MAF = how you build the agent brain and workflow
Foundry Agent Service = where you run, govern, expose, scale, and monitor it
```

Question	Foundry Agent Service	Microsoft Agent Framework (MAF)
What is it?	Managed cloud agent platform/runtime.	Code-first SDK/framework for building agent apps and workflows.
Main role	Host, scale, secure, observe, and expose agents.	Define orchestration logic, workflows, agent collaboration, tools, memory, and middleware.
Multi-agent style	Platform-managed agents, connected agents, hosted agents, tool-based delegation.	Code-defined multi-agent workflows: sequential, concurrent, handoff, group collaboration, durable workflows.
Best for	Enterprise deployment, managed endpoints, identity, RBAC, tracing, model/tool hosting.	Complex orchestration logic, custom routing, long-running workflows, testable code, provider flexibility.
Relationship	Foundry can host agents and expose Responses API/tools/runtime.	MAF agents can run locally, in your backend, or as Hosted Agents in Foundry.

3. Foundry Agent Service: Platform-First Multi-Agent

In Foundry Agent Service, the center of gravity is the managed runtime. You create agents with a model, instructions, tools, state, access control, tracing, and endpoint exposure. Foundry manages much of the lifecycle and operational platform responsibility.

- Agent lifecycle
- Conversations, threads, tool calls, and runs
- Scaling
- Microsoft Entra identity and RBAC
- Content filters and network isolation
- Built-in tools such as file search, code interpreter, memory, web search, MCP servers, and custom functions
- Observability and Application Insights integration

```
User / Application
      |
```

```
      v
Primary Foundry Agent
      |
      |-- Specialist Agent A
      |-- Specialist Agent B
      |-- Specialist Agent C
      |
      v
Final response
```

Typical examples include an HR policy router agent delegating to payroll, recruitment, compliance, or employee-relations agents; or a support agent delegating to billing, technical support, and account management agents.

4. Microsoft Agent Framework: Code-First Multi-Agent Orchestration

MAF is not primarily a cloud hosting platform. It is a developer framework for building agent systems in code. With MAF, you own the orchestration logic and can explicitly control which agent runs first, which agents run in parallel, when to retry, when to escalate, and how state is passed between agents.

- Agents
- Tools
- Chat clients
- Workflows
- Context providers
- Middleware
- Memory and persistence
- MCP integrations
- Human-in-the-loop control
- Durable execution
- Observability hooks

```
Application Code
      |
      v
MAF Orchestrator / Workflow
      |
      |-- Agent A: Research
      |-- Agent B: Critique
      |-- Agent C: Writer
      |-- Agent D: Validator
      |
      v
Business Output
```

5. Architecture Patterns

5.1 Foundry-only multi-agent pattern

```
Client App
   |
```

```
   v
Foundry Responses API
   |
   v
Primary Agent
   |
   |-- Built-in tool call
   |-- File search
   |-- Code interpreter
   |-- MCP server
   |-- Connected/specialist agent
   |
   v
Response
```

5.2 MAF multi-agent pattern

```
Client App
   |
   v
MAF Workflow / Orchestrator
   |
   |-- Agent 1
   |-- Agent 2
   |-- Agent 3
   |-- Human approval
   |-- Business rules
   |-- Durable state
   |
   v
Response / Action
```

5.3 MAF hosted in Foundry

```
Client App
   |
   v
Foundry Agent Endpoint
   |
   v
Hosted Agent Container
   |
   v
MAF Workflow
   |
   |-- Specialist Agent A
   |-- Specialist Agent B
   |-- Specialist Agent C
   |
   v
Foundry Observability / Identity / Scaling
```

A strong production pattern is to use MAF for orchestration and Foundry for hosting, governance, identity, tracing, model access, and endpoints.

6. LLM-to-Tool Traffic

In an agent system, the model usually does not call tools directly. Instead, the runtime or orchestrator manages a loop where the model proposes a tool call, the runtime executes the tool, and the result is returned to the model as context.

```
User request
   ↓
Agent runtime / orchestrator
   ↓
LLM call
   ↓
LLM returns tool-call intent
   ↓
Runtime/orchestrator executes tool
   ↓
Tool result returned to LLM
   ↓
LLM continues reasoning
   ↓
Final answer
```

Traffic includes prompt/context sent to the model, model output, tool-call metadata, tool input arguments, tool responses, follow-up LLM calls, telemetry, memory updates, and inter-agent messages.

7. Foundry Traffic Pattern

In Foundry Agent Service, much of the LLM-to-tool traffic is handled inside the managed runtime. The client application may only see the initial request and final response, while Foundry internally manages several LLM-tool-LLM loops.

```
Client App
   |
   | 1. User request
   v
Foundry Responses API / Agent Runtime
   |
   | 2. Prompt + thread context
   v
LLM
   |
   | 3. Tool call request
   v
Foundry Agent Runtime
   |
   | 4. Executes tool
   v
Tool / External System
   |
   | 5. Tool result
   v
Foundry Runtime
   |
   | 6. Sends result back to LLM
   v
```

```
LLM
   |
   | 7. Final or next tool call
   v
Foundry Runtime
   |
   | 8. Response to app
   v
Client App
```

Important training point: with Foundry-native agents, your application usually does not see every internal LLM/tool roundtrip. You rely on Foundry traces, run metadata, and observability to inspect what happened.

8. MAF Traffic Pattern

With MAF, your application or workflow usually sits in the middle. The orchestration code controls what context is sent, which tools are called, how results are filtered, and which agent receives which state.

```
Client App
   |
   | 1. User/task request
   v
MAF Workflow / Orchestrator
   |
   | 2. Prompt + workflow state
   v
LLM Provider
   |
   | 3. Tool call request
   v
MAF Runtime / Agent Code
   |
   | 4. Executes local/custom tool
   v
Tool / API / DB / File / MCP Server
   |
   | 5. Tool result
   v
MAF Runtime
   |
   | 6. Adds result to context
   v
LLM Provider
   |
   | 7. Continues reasoning
   v
MAF Runtime
   |
   | 8. Routes to next agent or final output
   v
Client App
```

Traffic area	Foundry Agent Service	MAF
Client to agent	App calls Foundry endpoint/API.	App calls your MAF service/function.

LLM calls	Managed by Foundry runtime.	Usually controlled by your MAF code/provider.
Tool calls	Managed by Foundry runtime for built-in/platform tools.	Executed by MAF tool functions, MCP clients, APIs, or plugins.
Tool result handling	Foundry decides how result is fed back to model.	You decide how much result goes back to model.
Inter-agent traffic	Managed through Foundry agent patterns.	Explicit in workflow code.
Observability	Built into Foundry and Application Insights integration.	Framework hooks and OpenTelemetry/traces can be used.
Data minimization	Controlled through Foundry configuration/tools.	Controlled directly in code.

9. How Much Traffic Is Generated?

A single user request may trigger multiple model calls and tool calls. For example, a request such as "Review this HR document package and find missing documents" could involve:

4. LLM call to understand the task
5. Tool call to search the folder
6. LLM call to inspect the results
7. Tool call to read document metadata
8. LLM call to decide missing documents
9. Tool call to query policy
10. LLM call to compare against policy
11. Final answer

This could result in four LLM calls, three tool calls, multiple internal messages, retrieved document chunks, and telemetry events.

10. Multi-Agent Traffic Explosion Problem

The biggest risk in multi-agent systems is context multiplication.

```
Bad pattern:
Agent A receives 40 KB context
Agent B receives same 40 KB + A output
Agent C receives same 40 KB + A + B
Agent D receives same 40 KB + A + B + C
```

This creates high token cost, slower latency, more chance of confusion, larger traces, and more sensitive data exposure.

```
Better pattern:
Agent A receives only intake data
Agent B receives search query + relevant metadata
Agent C receives selected evidence only
Agent D receives final structured facts and citations
```

11. Traffic Control Techniques

11.1 Foundry traffic control

- Reduce the number of exposed tools.
- Use fewer coarse-grained tools instead of many narrow tools when appropriate.
- Use specialist agents only when they add clear value.
- Limit retrieved chunks with top-k, metadata filters, document type filters, date filters, security trimming, and reranking.
- Keep tool responses structured and compact.
- Use Foundry observability to inspect decisions and tool usage.

```
Better tool result:
{
  "policy_name": "Termination Policy",
  "relevant_clause": "Notice period is 3 months...",
  "source": "HR-Termination-Policy.pdf",
  "page": 7,
  "confidence": 0.91
}
```

11.2 MAF traffic control

- Pass structured state rather than full chat history.
- Use agent-specific context.
- Use parallel agents carefully because they can increase total token traffic.
- Compress intermediate results.
- Use deterministic code between agents for routing and business rules.
- Use smaller or cheaper models for simple steps where appropriate.
- Cache tool results when possible.

```
Compact state object example:
{
  "case_type": "termination",
  "employee_country": "Norway",
  "relevant_documents": ["employment_contract", "termination_policy"],
  "open_questions": ["notice period", "missing signature"]
}
```

12. Cost and Latency Implications

12.1 Foundry-native cost and latency drivers

- Number of model calls
- Model type
- Tokens from retrieved data
- Number of tool calls
- Code interpreter or file processing activity
- Web search or external tool usage
- Connected-agent calls

- Search/retrieval latency
- Tool execution time
- Number of internal tool loops

12.2 MAF cost and latency drivers

- Number of agent calls
- How much context each agent receives
- Whether agents run sequentially or in parallel
- Tool result size
- Retry loops
- Critic/revision loops
- Memory and persistence reads/writes
- Workflow depth
- External API calls
- Approval steps

13. Rule of Thumb for Estimating Traffic

```
Total LLM traffic =
  number_of_agents
  × average_calls_per_agent
  × average_context_size

Total tool traffic =
  number_of_tool_calls
  × average_tool_request_size
  + number_of_tool_calls
  × average_tool_response_size

Inter-agent traffic =
  number_of_handoffs
  × average_state_payload_size
```

The expensive part is usually not the initial user message. The expensive part is retrieved context, repeated agent calls, and tool results passed back into the model.

14. Recommendations for Training and Production Design

- Use Foundry Agent Service when you need a managed enterprise agent platform.
- Use MAF when you need code-first orchestration, custom workflows, and precise traffic control.
- Use MAF hosted in Foundry when you need both complex orchestration and enterprise governance.
- Do not let every agent see every document and every previous message.
- Use structured state, filtered evidence, compact tool responses, and deterministic routing.
- Design agents around clear responsibilities and explicit data boundaries.

```
Recommended production pattern:
User request
   ↓
Router / Classifier
   ↓
Minimal state object
```

```
   ↓
Only relevant specialist agent
   ↓
Tool calls with filtered results
   ↓
Evidence summary
   ↓
Critic/validator only if needed
   ↓
Final answer
```

15. Sources and References Used in the Conversation

- Microsoft Learn: What is Microsoft Foundry Agent Service? — https://learn.microsoft.com/en-us/azure/foundry/agents/overview
- Microsoft Agent Framework documentation — https://learn.microsoft.com/en-us/agent-framework/
- Microsoft Agent Framework GitHub repository — https://github.com/microsoft/agent-framework
- Microsoft Agent Framework Build 2026 announcement — https://devblogs.microsoft.com/agent-framework/microsoft-agent-framework-at-build-2026-announce/